Cache Eviction Algorithms

08 October 2026 08:32 PM

Introduction

Caching makes applications faster, reduces database load, and improves user experience. However, cache memory is limited, so when the cache becomes full, it must decide which items to remove. Cache eviction strategies define how the cache chooses which items to remove to make room for new data .
Pasted markdown.md

1. LRU — Least Recently Used

LRU removes the item that has not been used for the longest time. It assumes that if an item has not been accessed recently, it is less likely to be needed again soon.
LRU tracks the order in which items are accessed. It can use a doubly linked list or a combination of a hashmap and a queue t o maintain this order.
When an item is accessed and found in the cache, it is moved to the Most Recently Used (MRU) position. When a new item arrive s and the cache is full, the Least Recently Used item is removed.
Example: Cache capacity = 3

Operation	Cache state, from least to most recently used
Start	[]
Add A	[A]
Add B	[A, B]
Add C	[A, B, C]
Access A	[B, C, A]
Add D	[C, A, D]

When D is added, B is evicted because B has not been accessed for the longest time.
Advantages: LRU is intuitive and efficient, and it works well for access patterns such as web browsing and API requests.
Disadvantages: It needs metadata to track access order, maintaining that order adds computational overhead, and it does not a dapt to every workload.
Source:
Pasted markdown.md

2. LFU — Least Frequently Used

LFU removes the item that has been accessed the fewest times. Unlike LRU, which focuses on when an item was last accessed, LF U focuses on how often an item has been accessed.
LFU maintains a frequency count for each cached item. Whenever an item is accessed, its frequency count increases.
When a new item arrives and the cache has space, it is added with a frequency count of 1. If the cache is full, the item with the lowest frequency is evicted. If multiple items have the same lowest frequency, a secondary strategy such as LRU or FIFO can be used to decide which item to remove.
Example: Cache capacity = 3

Operation	Cache state
Add A, B, C	A=1, B=1, C=1
Access A	A=2, B=1, C=1
Add D	B is evicted; A=2, C=1, D=1
Access C	A=2, C=2, D=1

When D is added, B is evicted because B has the lowest frequency. A remains because it has been accessed more often.
Advantages: LFU works well when access patterns are predictable and certain items remain popular over time.
Disadvantages: It needs extra metadata to maintain frequency counts, updating the counts adds overhead, and it may retain ite ms that were popular in the past but are no longer relevant.
Source:
Pasted markdown.md

3. FIFO — First In, First Out

FIFO removes the item that was added to the cache first. It does not consider how recently or how frequently an item was acce ssed.
Items are added to the end of a queue. When the cache becomes full, the item at the front of the queue —the oldest inserted item—is removed. Accessing an existing item does not change its position in the queue.
Example: Cache capacity = 3

Operation	Cache state, oldest to newest
Add A, B, C	[A, B, C]
Add D	[B, C, D] — A is evicted
Access B	[B, C, D] — order stays the same
Add E	[C, D, E] — B is evicted

Even though B was accessed before E was added, B is still removed because it was inserted earlier than C, D, and E.
Advantages: FIFO is simple to implement, has low overhead, and produces predictable eviction behavior.
Disadvantages: It ignores access patterns and may evict an old item that is still frequently used.
Source:
Pasted markdown.md

4. RR — Random Replacement

Random Replacement removes a randomly selected item when the cache is full. Unlike LRU, LFU, and FIFO, it does not track rece ncy, frequency, or insertion order for eviction decisions.
When an item is found in the cache, it is returned without changing the cache's ordering. When a new item arrives and the cac he is full, one of the existing items is selected randomly and replaced.
Example: Cache capacity = 3

Operation	Example cache state
Add A, B, C	[A, B, C]
Add D	[B, C, D] — A is randomly evicted
Add E	[C, E, D] — B is randomly evicted

These eviction choices are illustrative. Actual results can differ because the victim is chosen randomly.
Advantages: Random Replacement is simple, requires little overhead, and can be suitable for unpredictable access patterns.
Disadvantages: It may randomly remove frequently used items, performs poorly for some stable access patterns, and can result in a low cache hit rate.
Source:
Pasted markdown.md

5. MRU — Most Recently Used

MRU is the opposite of LRU. Instead of removing the item that has not been used for the longest time, MRU removes the item th at was accessed most recently when the cache is full.
This strategy assumes that the most recently used item may be temporary and is less likely to be needed again soon. In some w orkloads, older items may be more valuable to retain.
When an item is accessed, it becomes the most recently used item. When a new item arrives and the cache is full, that most re cently used item is evicted.
Example: Cache capacity = 3

Operation	Cache state
Add A, B, C	[A, B, C]
Access C	C becomes the most recently used item
Add D	[A, B, D] — C is evicted
Access B	B becomes the most recently used item
Add E	[A, D, E] — B is evicted

Advantages: MRU can be useful for particular workloads where retaining older data is more beneficial. It can also be implemen ted with relatively simple metadata.
Disadvantages: It is usually suboptimal for common access patterns where recently used items are likely to be reused. It is l ess commonly used than LRU or LFU.
Source:
Pasted markdown.md

6. TTL — Time to Live

TTL removes cached items after a specified amount of time. Each item has a lifespan, and once that time expires, the item is considered expired regardless of how frequently it has been accessed.
TTL helps keep cached data fresh and prevents outdated entries from remaining in the cache indefinitely.
When an item is inserted, its expiration time is set, often by adding the TTL duration to the current time. When an item is a ccessed, the cache checks whether it has expired. If it has expired, it is removed and the request becomes a cache miss. Otherwise, the request is a cache hit.
Expired items may be cleaned up periodically or removed when they are accessed.
Example:
Suppose the cache uses these expiration settings:

- Item A has a TTL of 5 seconds.
- Item B has a TTL of 10 seconds.
- After 6 seconds, A has expired, while B has 4 seconds remaining.
- If item C is then added with a TTL of 5 seconds, C will have its own expiration time.

Simple flow:

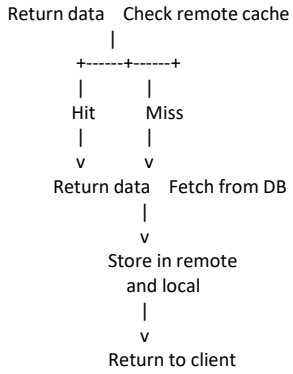
```
Insert item
   |
   v
Set expiration time
   |
   v
Item accessed?
   |
   v
Check expiration
   |
   +---- Expired ----> Remove item
   |                   Cache miss
   |
   +---- Not expired -> Return item
                        Cache hit
```

Redis and Memcached support expiration times for individual keys.
Advantages: TTL helps maintain data freshness, is easy to configure, has relatively low overhead, and prevents expired entrie s from remaining indefinitely.
Disadvantages: A frequently accessed item may expire too early, an irrelevant item may occupy space until its TTL expires, an d TTL does not adapt automatically to changing access patterns.
Source:
Pasted markdown.md

7. Two-Tiered Caching

Two-Tiered Caching combines a local in-memory cache with a remote, shared cache. The application checks the local cache first because it is faster. If the item is n ot found there, the application checks the remote cache.
The first tier is the local cache, which runs in memory on the same server as the application. It provides very fast access t o frequently used, or “hot,” data. Examples include a HashMap, LRUCache, and Guava Cache.
The second tier is the remote cache, which is shared across servers. It can provide larger storage capacity and shared access to cached data, but accessing it involves network overhead. Redis and Memcached are examples.
If the item is missing from both caches, the application fetches it from the primary data source, such as a database, and sto res it in both cache tiers before returning the result to the client.
Request flow:

```
      Client request
           |
           v
     Check local cache
           |
     +-----+-------+
     |             |
    Hit           Miss
     |             |
     v             v
```



Advantages: Local cache hits provide very fast responses. The remote cache provides shared storage across servers and helps r educe database load. It can also act as a fallback if the local cache fails.

Disadvantages: Maintaining two cache tiers introduces synchronization and consistency complexity. Data can become stale betwe en the local and remote caches, and remote cache hits are slower than local cache hits.